

Documentacao da API

Esta API e um backend Express com autenticacao JWT, usuarios, amizades com fluxo de solicitacao/aprovacao e posts.

Base URL

`http://localhost:3000`

Autenticacao

As rotas protegidas usam token JWT no header:

`Authorization: Bearer <accessToken>`

O `accessToken` e retornado no login/cadastro e tambem pode ser renovado usando o `refreshToken`.

Modelos Principais

Usuario

Campos principais retornados pela API:

```
{
  "_id": "ObjectId",
  "name": "Nome do usuario",
  "email": "email@example.com",
  "imagem": "https://i.pravatar.cc/300?u=123",
  "role": "user",
  "friends": ["ObjectId"],
  "solicitacao": ["ObjectId"],
  "aprovacao": ["ObjectId"]
}
```

Regras:

- `password` e `refreshTokens` nao sao retornados nas respostas.
- `friends` guarda amizades ja aprovadas.
- `solicitacao` guarda usuarios para quem o usuario logado enviou pedido de amizade.
- `aprovacao` guarda usuarios que enviaram pedido de amizade para o usuario logado.
- Uma amizade aprovada deve existir dos dois lados: se A esta em `friends` de B, B tambem deve estar em `friends` de A.

Post

```
{
  "_id": "ObjectId",
  "title": "Titulo",
  "description": "Texto do post",
  "isPublic": true,
  "creationDate": "2026-01-01",
  "creationTime": "10:30:00",
  "createdBy": "ObjectId"
}
```

Regras:

- `creationDate`, `creationTime` e `createdBy` são preenchidos pelo servidor.
- O usuário só pode deletar posts criados por ele.
- Posts privados de amigos aparecem na listagem de posts visíveis.

Endpoints Gerais

GET /

Verifica se a API está ativa.

Autenticação: não requer.

Resposta:

```
Hello, World!
```

GET /api/data

Retorna um exemplo simples de resposta JSON com mensagem e data/hora atual.

Autenticação: não requer.

Resposta:

```
{
  "message": "This is some sample data from the API.",
  "timestamp": "2026-01-01T10:30:00.000Z"
}
```

Autenticação

POST /auth/register

Cadastra um novo usuário e já retorna tokens de acesso.

Autenticação: não requer.

Body obrigatório:

```
{
  "name": "Lucas",
  "email": "lucas@example.com",
  "password": "Senha123"
}
```

Body opcional:

```
{
  "image": "https://i.pravatar.cc/300?u=123"
}
```

Regras de negócio:

- `name`, `email` e `password` são obrigatórios.
- `email` deve ser único.
- A senha é criptografada antes de ser salva.
- Ao cadastrar, a API gera `accessToken` e `refreshToken`.
- O `refreshToken` gerado fica salvo no usuário para permitir renovação futura.

Resposta de sucesso:

```
{
  "message": "Usuario criado com sucesso",
  "accessToken": "jwt",
  "refreshToken": "jwt",
  "user": {
    "id": "ObjectId",
    "name": "Lucas",
    "email": "lucas@example.com",
    "role": "user"
  }
}
```

POST /auth/login

Autentica um usuario existente.

Autenticacao: nao requer.

Body obrigatorio:

```
{
  "email": "lucas@example.com",
  "password": "Senha123"
}
```

Regras de negocio:

- `email` e `password` sao obrigatorios.
- A API busca o usuario pelo email incluindo a senha, que normalmente fica oculta.
- A senha informada e comparada com a senha criptografada usando bcrypt.
- Se as credenciais forem validas, gera `accessToken` e `refreshToken`.
- O novo `refreshToken` e salvo no usuario.

POST /auth/refresh

Gera um novo `accessToken` usando um `refreshToken`.

Autenticacao: nao requer header Bearer; usa o refresh token no body.

Body obrigatorio:

```
{
  "refreshToken": "jwt"
}
```

Regras de negocio:

- O `refreshToken` e obrigatorio.
- O token precisa estar valido, nao expirado e assinado com o segredo correto.
- O usuario dono do token precisa existir.
- O token precisa estar salvo na lista `refreshTokens` do usuario.
- Se o token nao estiver salvo, a renovacao e recusada.

Resposta de sucesso:

```
{
  "message": "Access token renovado com sucesso",
  "accessToken": "jwt"
}
```

```
}
```

POST /auth/logout

Encerra uma sessao especifica removendo o refresh token enviado.

Autenticacao: nao requer header Bearer; usa o refresh token no body.

Body obrigatorio:

```
{
  "refreshToken": "jwt"
}
```

Regras de negocio:

- O `refreshToken` e obrigatorio.
- O token e validado para descobrir o usuario dono.
- Se o usuario existir, o token e removido da lista `refreshTokens`.
- Outros refresh tokens do mesmo usuario continuam validos.

POST /auth/logout-all

Encerra todas as sessoes do usuario.

Autenticacao: regra de negocio depende do usuario autenticado em `req.userId`.

Body: nao requer.

Regras de negocio:

- Busca o usuario autenticado.
- Limpa todos os tokens da lista `refreshTokens`.
- Depois disso, nenhum refresh token antigo consegue renovar acesso.

Observacao: esta rota usa `req.userId`; portanto deve ser chamada em um fluxo autenticado.

Usuarios

Todas as rotas abaixo usam:

```
Authorization: Bearer <accessToken>
```

GET /api/users

Lista todos os usuarios cadastrados.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Apenas usuarios autenticados podem acessar.
- Retorna os usuarios sem `password` e sem `refreshTokens`.

GET /api/users/:id

Busca um usuario pelo id.

Parametro de rota:

Nome	Tipo	Obrigatorio	Descricao
<code>id</code>	string	sim	ObjectId do usuario

Body: nao requer.

Regras de negocio:

- O usuario precisa existir.
- Se nao existir, retorna erro.

POST /api/users

Cria um usuario pela area protegida da API.

Body obrigatorio:

```
{
  "name": "Lucas",
  "email": "lucas@example.com",
  "password": "Senha123"
}
```

Body opcional:

```
{
  "imagem": "https://i.pravatar.cc/300?u=123"
}
```

Regras de validacao:

- `name`: minimo 2 caracteres, maximo 100.
- `email`: formato de email valido.
- `password`: minimo 8 caracteres, precisa ter pelo menos uma letra maiuscula e um numero.
- `imagem`: se enviada, deve ser uma URL valida.

Regras de negocio:

- Email nao pode estar cadastrado.
- Senha e criptografada antes de salvar.

GET /api/users/availablefriends

Lista usuarios que ainda podem receber pedido de amizade do usuario logado.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Exclui o proprio usuario logado.
- Exclui quem ja esta em `friends`.
- Exclui quem ja esta em `solicitacao`.
- Exclui quem ja esta em `aprovacao`.

GET /api/users/friends

Lista os amigos do usuario logado.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Busca o usuario logado.
- Retorna o campo `friends` populado com dados basicos dos amigos, como `name` e `email`.

POST /api/users/add_friend

Envia um pedido de amizade.

Body obrigatorio:

```
{
  "friendId": "ObjectId"
}
```

Regras de negocio:

- `friendId` e obrigatorio.
- O usuario logado nao pode enviar pedido para si mesmo.
- O `friendId` precisa existir.
- Se o usuario ja esta em `friends`, nao pode enviar pedido.
- Se o usuario ja esta em `solicitacao`, o pedido ja foi enviado.
- Se o usuario ja esta em `aprovacao`, significa que ele ja enviou pedido para o usuario logado.
- Quando o pedido e criado:
- `friendId` entra em `solicitacao` do usuario logado.
- id do usuario logado entra em `aprovacao` do usuario convidado.

GET /api/users/solicitacao

Lista pedidos de amizade enviados pelo usuario logado.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Retorna usuarios que estao na lista `solicitacao`.
- Esses sao pedidos enviados pelo usuario logado que ainda aguardam aprovacao do outro usuario.

GET /api/users/aprovacao

Lista pedidos de amizade recebidos pelo usuario logado.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Retorna usuarios que estao na lista `aprovacao`.
- Esses usuarios enviaram pedido de amizade para o usuario logado.
- O usuario logado pode aprovar ou reprovar esses pedidos.

POST /api/users/aprovacao

Aprova um pedido de amizade recebido.

Body obrigatorio:

```
{
  "friendId": "ObjectId"
}
```

Regras de negocio:

- `friendId` e obrigatorio.
- O usuario logado nao pode aprovar a si mesmo.
- O `friendId` precisa existir.
- O `friendId` precisa estar em `aprovacao` do usuario logado.
- O id do usuario logado precisa estar em `solicitacao` do usuario que enviou o pedido.
- Ao aprovar:
 - `friendId` sai de `aprovacao` do usuario logado.
 - `friendId` entra em `friends` do usuario logado.
 - id do usuario logado sai de `solicitacao` do usuario solicitante.
 - id do usuario logado entra em `friends` do usuario solicitante.
- A amizade fica bilateral.

POST /api/users/reprovacao

Reprova um pedido de amizade recebido.

Body obrigatorio:

```
{
  "friendId": "ObjectId"
}
```

Regras de negocio:

- `friendId` e obrigatorio.
- O usuario logado nao pode reprovar a si mesmo.
- O `friendId` precisa existir.
- O `friendId` precisa estar em `aprovacao` do usuario logado.
- O id do usuario logado precisa estar em `solicitacao` do usuario que enviou o pedido.
- Ao reprovar:
 - `friendId` sai de `aprovacao` do usuario logado.
 - id do usuario logado sai de `solicitacao` do usuario solicitante.
 - nenhum dos usuarios e adicionado em `friends`.

Posts

Todas as rotas abaixo usam:

```
Authorization: Bearer <accessToken>
```

POST /api/post

Cria um post para o usuario logado.

Body obrigatorio:

```
{
  "title": "Titulo do post",
  "description": "Texto do post"
}
```

Body opcional:

```
{
  "isPublic": true
}
```

Regras de validacao:

- **title**: obrigatorio, minimo 2 caracteres, maximo 200.
- **description**: obrigatorio, minimo 1 caractere.
- **isPublic**: opcional, booleano, padrao **true**.

Regras de negocio:

- **creationDate** e preenchido pelo servidor com a data atual.
- **creationTime** e preenchido pelo servidor com a hora atual.
- **createdBy** recebe o id do usuario logado.
- O cliente nao deve controlar autor, data ou hora.

GET /api/post

Lista posts criados pelo usuario logado.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Retorna apenas posts cujo **createdBy** e o usuario logado.
- Os posts sao ordenados do mais recente para o mais antigo.
- O autor e populado com **name** e **email**.

GET /api/post/list

Lista posts visiveis para o usuario logado.

Parametros: nenhum.

Body: nao requer.

Regras de negocio:

- Retorna posts publicos.
- Retorna posts criados pelo proprio usuario logado.
- Retorna posts privados criados por amigos do usuario logado.
- Posts privados de usuarios que nao sao amigos nao aparecem.
- Os posts sao ordenados do mais recente para o mais antigo.

DELETE /api/post/:id

Remove um post.

Parametro de rota:

Nome	Tipo	Obrigatorio	Descricao
id	string	sim	ObjectId do post

Body: nao requer.

Regras de validacao:

- [id](#) deve ter 24 caracteres.

Regras de negocio:

- O post precisa existir.
- Somente o usuario que criou o post pode remove-lo.
- Se outro usuario tentar remover, a API retorna erro de permissao.

Padrao de Erros

A API usa um middleware central para converter erros em respostas HTTP.

Exemplos de regras:

- Erro de validacao do Mongoose: [422](#).
- ObjectId invalido do Mongoose: [400](#).
- Registro duplicado, como email repetido: [409](#).
- Erros de negocio com `AppError`: usam o `statusCode` informado.
- Erro inesperado: [500](#).

Formato comum:

```
{
  "success": false,
  "message": "Mensagem do erro"
}
```

Quando o erro vem do Zod, a resposta pode trazer uma lista de campos:

```
{
  "success": false,
  "errors": [
```

```
{
  "field": "body.email",
  "message": "Email invalido"
}
]
```